

Stack & Queue Part 2*Striver SDE Sheet - Precise Revision Notes*

This document covers the advanced Stack & Queue problems: caches, histogram, sliding window, BFS, stock span, and celebrity-style elimination.

Quick Pattern Map

Problem Type	Best Structure	Remember
LRU Cache	HashMap + Doubly Linked List	Most recent near head, least recent near tail
LFU Cache	HashMap + Frequency Lists	Remove least frequency, then least recent
Histogram / Span	Monotonic Stack	Find nearest smaller/greater boundary
Sliding Window Maximum	Deque	Front stores max candidate
Rotten Oranges	BFS Queue	Multi-source level traversal

1. LRU Cache

Problem: Design get(key) and put(key, value) in $O(1)$, removing the Least Recently Used item when capacity is full.

Core idea: Use HashMap for $O(1)$ access and Doubly Linked List for $O(1)$ delete/insert and usage order.

Approach

1. Create dummy head and tail nodes.
2. Most recently used node is placed after head.
3. Least recently used node is before tail.
4. On get, return value and move node to front.
5. On put, update existing node or insert new node at front.
6. If capacity is exceeded, remove node before tail.

Important Notes

- HashMap stores key $\rightarrow$ node.
- DLL is needed because removing any node must be $O(1)$.

Complexity

Get: $O(1)$ | Put: $O(1)$ | Space: $O(\text{capacity})$

2. LFU Cache

Problem: Design cache that removes the Least Frequently Used item. If frequencies tie, remove the least recently used among them.

Core idea: Use key-node map, frequency-list map, and a minFreq variable.

Approach

1. Map every key to its node.
2. Map every frequency to a linked list of nodes with that frequency.
3. On get, increase node frequency and move it to the next frequency list.
4. On put existing key, update value and increase frequency.
5. On put new key when full, remove least recent node from minFreq list.
6. Insert new key with frequency 1 and set minFreq = 1.

Important Notes

- minFreq quickly tells which frequency group to delete from.
- Inside the same frequency, DLL/list maintains recency.

Complexity

Get: $O(1)$ | Put: $O(1)$ | Space: $O(\text{capacity})$

3. Largest Rectangle in Histogram

Problem: Find the maximum rectangular area possible in a histogram.

Core idea: For each bar, find the largest range where it is the minimum height using a monotonic increasing stack.

Approach

1. Traverse bars from left to right.
2. Maintain stack of indices with increasing heights.
3. When current height is smaller than stack top height, pop and calculate area.
4. For popped bar, current index is right smaller boundary.
5. New stack top after popping is left smaller boundary.
6. After traversal, process remaining stack.

Important Notes

- Area = height * width.
- Width = rightSmall - leftSmall - 1.

Key Formula / Template

area = heights[top] * (rightSmall - leftSmall - 1)

Complexity

Brute force: $O(n^2)$ | Optimal time: $O(n)$ | Space: $O(n)$

4. Sliding Window Maximum

Problem: Find the maximum value in every window of size k.

Core idea: Use a deque storing indices in decreasing value order. The front always contains the maximum index of current window.

Approach

1. Remove front indices that are outside the current window.
2. Remove back indices while their values are $\leq$ current value.
3. Push current index at the back.
4. When $i \geq k - 1$, answer is $\text{arr}[\text{dq.front}()]$.

Important Notes

- Deque stores only useful candidates.
- Older smaller elements are removed because they can never become maximum.

Complexity

Brute force: $O(n*k)$ | **Optimal time:** $O(n)$ | **Space:** $O(k)$

5. Implement Min Stack

Problem: Design stack operations push, pop, top, and getMin in $O(1)$.

Core idea: Common approach stores each value with the minimum so far. Optimized approach uses encoding and one min variable.

Approach

1. Pair stack approach: push (value, currentMin).
2. For getMin, return the second value of top pair.
3. Space optimized approach: when $x < \text{min}$, push encoded value $2*x - \text{min}$ and update $\text{min} = x$.
4. While popping encoded value, recover previous minimum using $2*\text{min} - \text{encoded}$.

Important Notes

- Use long for encoding to avoid integer overflow.
- Pair stack is easier and safer for interviews.

Key Formula / Template

$\text{encoded} = 2*x - \text{min}$

$\text{previousMin} = 2*\text{min} - \text{encoded}$

Complexity

All operations: $O(1)$ | **Pair stack space:** $O(n)$ | **Optimized extra space:** $O(1)$

6. Rotten Oranges

Problem: Find the minimum time needed to rot all fresh oranges in a grid.

Core idea: This is a multi-source BFS problem. Start BFS from all initially rotten oranges together.

Approach

1. Push all rotten oranges into queue.
2. Count total fresh oranges.
3. Run BFS level by level.

4. For each rotten orange, rot its fresh neighbors in four directions.
5. Increase time after each level where at least one orange rots.
6. If fresh count becomes zero, return time; otherwise return -1.

Important Notes

- All rotten oranges spread infection at the same time.
- Level count in BFS represents minutes.

Complexity

Time: $O(n*m)$ | **Space:** $O(n*m)$

7. Stock Span Problem

Problem: For each day, find consecutive previous days with price $\leq$ current price.

Core idea: Use a monotonic decreasing stack of indices to find previous greater price.

Approach

1. Traverse prices from left to right.
2. While stack is not empty and price[stack.top] $\leq$ current price, pop.
3. If stack is empty, span = $i + 1$.
4. Otherwise span = $i - \text{stack.top}$.
5. Push current index.

Important Notes

- Previous greater element breaks the span.
- Similar to nearest greater to left.

Key Formula / Template

span = stack.empty() ? $i + 1$: $i - \text{stack.peek}()$

Complexity

Brute force: $O(n^2)$ | **Optimal time:** $O(n)$ | **Space:** $O(n)$

8. Maximum of Minimums of Every Window Size

Problem: For every window size from 1 to n , find the maximum among all window minimums.

Core idea: For each element, find the window length in which it can be the minimum using previous smaller and next smaller.

Approach

1. Find previous smaller index for every element.
2. Find next smaller index for every element.
3. For each index i , compute length = nextSmaller[i] - prevSmaller[i] - 1.
4. Update ans[length] = max(ans[length], arr[i]).
5. Fill missing values from right to left: ans[i] = max(ans[i], ans[$i+1$]).

Important Notes

- An element can be the minimum only until a smaller element appears on left or right.
- Final fill handles window sizes not directly updated.

Key Formula / Template

```
length = nextSmaller[i] - prevSmaller[i] - 1
ans[i] = max(ans[i], ans[i + 1])
```

Complexity**Time: $O(n)$ | Space: $O(n)$** **9. The Celebrity Problem****Problem:** Find a person who knows nobody but is known by everybody.**Core idea:** Use elimination. If A knows B, A cannot be celebrity. Otherwise B cannot be celebrity.**Approach**

1. Set two pointers: $i = 0$ and $j = n - 1$.
2. If i knows j , increment i because i cannot be celebrity.
3. Else decrement j because j cannot be celebrity.
4. One candidate remains after elimination.
5. Verify candidate: candidate knows nobody and everyone knows candidate.

Important Notes

- Elimination only finds a candidate; verification is compulsory.
- Celebrity row should contain all 0s and celebrity column should contain all 1s except self.

Complexity**Brute force: $O(n^2)$ | Optimal time: $O(n)$ | Space: $O(1)$** **Final Templates****Deque Sliding Window Maximum**

```
for (int i = 0; i < n; i++) { while (!dq.isEmpty() && dq.peekFirst() <= i-k) dq.pollFirst(); while
(!dq.isEmpty() && arr[dq.peekLast()] <= arr[i]) dq.pollLast(); dq.offerLast(i); if (i >= k-1)
ans.add(arr[dq.peekFirst()]); }
```

BFS Level Traversal

```
while (!q.isEmpty()) { int size = q.size(); for (int i = 0; i < size; i++) { node = q.poll(); process
neighbours; } time++; }
```